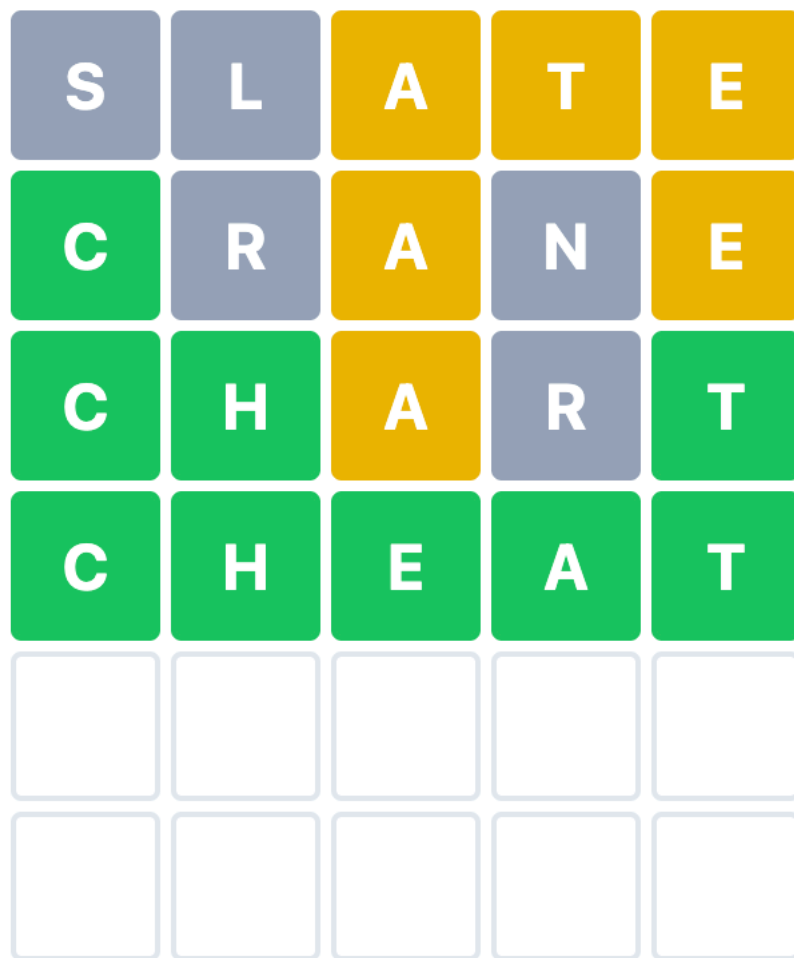


Bot or Not? Using Machine Learning Models to Distinguish Assisted and Unassisted Wordle Play

Pratyush Rallapally

March 17, 2026



Introduction

Every day at midnight, without fail, my friend group completes the daily Wordle, and we all post our daily summaries, listing our guesses of the day and comparing who completed it in the least amount of guesses.

And every day, by 12:15, without fail, one of my friends inevitably gets the word in two or three.

He claims he's "just lucky".

I'm not sure I buy it.



Wordle’s shareable format makes these kinds of comparisons inevitable. Because players only post the emoji summary rather than the actual guesses, the grid acts as a compressed record of the decision process behind each game. In principle, these summaries contain meaningful information about how a player arrived at the answer—how quickly they identified correct letters, how efficiently they narrowed down possibilities, and how their guesses improved over time. If someone were secretly using an external solver or looking up words, their patterns of guesses might look measurably different from someone solving the puzzle organically.

In this project, I’ll be investigating whether or not these Wordle result summaries can be used to classify outcomes statistically consistent with assisted (“invalid”) or unassisted (“valid”) play.

Wordle Basics

In late 2021, Wordle was publicly released, and amassed widespread acclaim and viral popularity after the introduction of a shareable emoji-based results format, and its sale to the New York Times.

The game is fairly simple, and works as follows:

- Players have six tries to guess a five-letter English word.
- After each guess, the letters are color-coded based on accuracy: green for the right letter in the right spot, yellow for the right letter in the wrong spot, and grey for neither.

When one completes a game of Wordle, a summary of their game is presented in a shareable emoji summary, formatted like this:

“Wordle 1,670 4/6*
”

Despite their simplicity, there are lots of features that can be derived from these summaries, and these features will be instrumental to the recipes of the machine learning models later on.

Further Considerations

Early on in the project planning process, I ran into a fairly substantial roadblock: there was no way for me to know if public Wordle games were valid or not. Considering that my main goal was to classify data as valid/invalid, the models to be used required labeled data, which would be impossible to find.

As a solution, I decided to create my own synthetic dataset that was constructed to generate labeled examples of both assisted and unassisted play. This was done by simulating Wordle games under valid and invalid conditions using a custom Wordle simulation engine. Here is how they work, briefly:

Wordle Simulation Engine

At the core of the data generation process is a simulation framework that replicates the mechanics of Wordle. The engine operates on a list of valid five-letter words and enforces the same rules as the original game.

For each simulated game, a target solution word is selected, and guesses are generated sequentially. After each guess, feedback is computed using the standard Wordle rules:

- Green (2): correct letter in the correct position
- Yellow (1): correct letter in the wrong position
- Grey (0): letter not present

The engine tracks the set of remaining candidate solutions consistent with all previous feedback. After each guess, candidate words that would produce different feedback patterns are eliminated, leaving only those still consistent with the information revealed so far.

The simulation also maintains a record of the guess history and calculates the size of the candidate set before each guess. This allows the model to approximate the entropy of the remaining search space and measure how efficiently guesses reduce uncertainty about the solution.

At the end of each simulated game, a set of summary features is extracted from the guess history, including the number of guesses taken, the rate of entropy reduction, and the fraction of green and yellow letters revealed over time.

Below is the code for the functions that extract all features: (As an aside, the bulk of this project was getting this engine to be scalable).

```
library(tidyverse)
library(dplyr)
library(purrr)
library(furrr)
library(ggplot2)
library(yardstick)
library(tune)
library(knitr)
library(kableExtra)
library(tidymodels)

MAX_GUESSES <- 6
OPTIMAL_FIRST_GUESS <- "slate"

clean_word_list <- function(words) {
  words <- tolower(trimws(words))
  words <- words[nchar(words) == 5]
  words <- words[grepl("^[a-z]{5}$", words)]
  unique(words)
}
```

```

get_feedback_chars <- function(guess_chars, sol_chars) {
  feedback <- integer(5)
  used <- logical(5)

  for (i in 1:5) {
    if (guess_chars[i] == sol_chars[i]) {
      feedback[i] <- 2
      used[i] <- TRUE
    }
  }

  for (i in 1:5) {
    if (feedback[i] == 0) {
      for (j in 1:5) {
        if (!used[j] && guess_chars[i] == sol_chars[j]) {
          feedback[i] <- 1
          used[j] <- TRUE
          break
        }
      }
    }
  }

  feedback
}

# Vectorized version: one guess vs many candidates at once
# Returns n x 5 matrix of feedback values
get_feedback_matrix <- function(guess_chars, candidates_matrix) {
  n <- nrow(candidates_matrix)
  feedback <- matrix(0L, nrow = n, ncol = 5)
  used <- matrix(FALSE, nrow = n, ncol = 5)

  # Pass 1: mark greens
  for (i in 1:5) {
    match <- candidates_matrix[, i] == guess_chars[i]
    feedback[match, i] <- 2L
    used[match, i] <- TRUE
  }

  # Pass 2: mark yellows
  for (i in 1:5) {
    need_yellow <- feedback[, i] == 0L
    if (!any(need_yellow)) next

    for (j in 1:5) {
      check <- need_yellow & !used[, j] & candidates_matrix[, j] == guess_chars[i]
      if (any(check)) {
        feedback[check, i] <- 1L
        used[check, j] <- TRUE
        need_yellow <- need_yellow & !check
      }
    }
  }
}

```

```

    }
  }

  feedback
}

filter_candidates <- function(candidates_idx, guess_idx, feedback, guesses_split, candidates_matrix = NA) {
  guess_chars <- guesses_split[[guess_idx]]

  # Use vectorized version if matrix provided
  if (!is.null(candidates_matrix)) {
    fb_matrix <- get_feedback_matrix(guess_chars, candidates_matrix)
    keep <- apply(fb_matrix, 1, function(row) all(row == feedback))
    return(candidates_idx[keep])
  }

  # Fallback to scalar version
  keep <- vapply(
    candidates_idx,
    function(i) {
      all(get_feedback_chars(guess_chars, guesses_split[[i]]) == feedback)
    },
    logical(1)
  )

  candidates_idx[keep]
}

init_state <- function() {
  list(
    greens = rep(NA_character_, 5),
    yellows = vector("list", 5),
    grays = character(0),
    used_letters = character(0)
  )
}

update_state <- function(state, guess_chars, feedback) {
  for (i in 1:5) {
    l <- guess_chars[i]
    state$used_letters <- union(state$used_letters, l)

    if (feedback[i] == 2) {
      state$greens[i] <- l
    } else if (feedback[i] == 1) {
      state$yellows[[i]] <- union(state$yellows[[i]], l)
    } else {
      state$grays <- union(state$grays, l)
    }
  }
  state
}

```

```

}

letter_freq <- function(candidates_idx, guesses_split) {
  letters <- unlist(guesses_split[candidates_idx])
  tab <- table(letters)
  as.numeric(tab / sum(tab)) |> setNames(names(tab))
}

score_valid_guess <- function(word_chars, state, lf) {
  novelty <- sum(!word_chars %in% state$used_letters)
  freq_score <- sum(lf[word_chars], na.rm = TRUE)
  green_score <- sum(!is.na(state$greens) & word_chars == state$greens)

  novelty * 0.6 + freq_score * 0.3 + green_score * 0.1
}

expected_entropy <- function(guess_idx, candidates_idx, guesses_split, candidates_matrix) {
  guess_chars <- guesses_split[[guess_idx]]
  fb_matrix <- get_feedback_matrix(guess_chars, candidates_matrix)

  # Convert rows to string keys for grouping
  fb_keys <- apply(fb_matrix, 1, paste, collapse = "")
  probs <- table(fb_keys) / length(fb_keys)
  -sum(probs * log2(probs))
}

# Fast batch entropy computation for all candidates
compute_all_entropies <- function(candidates_idx, candidates_matrix, guesses_split) {
  n <- length(candidates_idx)
  entropies <- numeric(n)

  for (k in seq_len(n)) {
    guess_chars <- guesses_split[[candidates_idx[k]]]
    fb_matrix <- get_feedback_matrix(guess_chars, candidates_matrix)
    # Pack feedback into single integer (0-242): faster than paste
    fb_key <- fb_matrix[,1] * 81L + fb_matrix[,2] * 27L + fb_matrix[,3] * 9L +
      fb_matrix[,4] * 3L + fb_matrix[,5]
    probs <- tabulate(fb_key + 1L, nbins = 243) / n
    probs <- probs[probs > 0]
    entropies[k] <- -sum(probs * log2(probs))
  }
  entropies
}

entropy <- function(n) log2(max(n, 1))

extract_features <- function(history) {
  entropies <- vapply(history, `[`, numeric(1), "entropy_before")

```

```

drops <- -diff(entropies)

# Fix pathological values
drops[!is.finite(drops)] <- 0

tibble(
  n_guesses = length(history),
  entropy_start = entropies[1],
  entropy_end = tail(entropies, 1),
  mean_entropy_drop = ifelse(length(drops) > 0, mean(drops), 0),
  max_entropy_drop = ifelse(length(drops) > 0, max(drops), 0),
  early_entropy_drop = ifelse(length(drops) >= 2,
                              mean(drops[1:2]),
                              0),
  monotone_entropy = ifelse(length(drops) > 0,
                             all(drops >= -1e-6),
                             TRUE),
  green_fraction = mean(vapply(history, \ (h) mean(h$feedback == 2), numeric(1))),
  yellow_fraction = mean(vapply(history, \ (h) mean(h$feedback == 1), numeric(1)))
)
}

```

Simulating Games

To encode the differences between “invalid” and “valid” in the synthetic data, I had to take some liberties in what I thought made a game suspicious. This ended up being a fun thought experiment, and these are the assumptions I came up with:

- Low guess counts are suspicious, since the odds of getting the right word out of thousands of options in so few guesses are outrageously unlikely
- Random explosions of greens are suspicious, especially if there are no yellows
- Despite this, humans are imperfect, and are much more likely to make non-optimal guesses (after all, how many times have you made guesses that realistically yielded no information?)
- Along those lines, bots are much more likely to make exclusively optimal guesses that prioritize information gain

With that out of the way, here’s a brief description of how these assumptions are represented in the invalid and valid algorithms.

“Invalid” Algorithm How it works:

The algorithm begins with the best starting word (“slate”), a commonly recommended optimal opening guess. After the first guess, it evaluates all remaining candidate words and selects the guess that maximizes the expected entropy reduction of the candidate solution set.

To do this, the algorithm simulates the feedback pattern that each possible guess would produce against all remaining candidate solutions. These feedback patterns define probability distributions over possible outcomes, allowing the algorithm to compute the expected information gain (entropy). The guess with the highest expected entropy is selected, as it maximizes the expected reduction in uncertainty.

This process continues until the correct word is found or the six-guess limit is reached.

This strategy approximates solver-based play by aggressively optimizing each guess to extract as much information as possible from the remaining search space.

“Valid” Algorithm How it works:

To simulate unassisted play, an additional algorithm generates guesses intended to resemble more typical human strategies.

Rather than explicitly maximizing information gain, this strategy evaluates candidate guesses using a scoring function based on three factors:

Letter novelty: preference for guesses containing letters that have not yet been tried

Letter frequency: preference for letters that appear frequently among remaining candidate words

Green letter reinforcement: preference for guesses that preserve letters already confirmed in the correct position

These components are combined into a weighted score used to rank possible guesses. The highest-scoring candidate is typically chosen as the next guess.

To further reflect imperfect human play, the algorithm occasionally selects a random guess from the remaining candidates with a 15% chance; in other words, there’s a 15% chance that the algorithm makes (at best) a lateral move, and doesn’t gain any useful information at all. This introduces variability and prevents the strategy from behaving too optimally.

Here’s the code for the simulation engine, using the functions defined prior:

```
simulate_game <- function(
  solution_idx,
  guesses_split,
  word_list,
  invalid = FALSE
) {
  candidates_idx <- seq_along(word_list)
  # Pre-build matrix of all candidate chars (n x 5)
  all_chars_matrix <- do.call(rbind, guesses_split)
  candidates_matrix <- all_chars_matrix[candidates_idx, , drop = FALSE]

  history <- list()
  state <- init_state()

  for (g in seq_len(MAX_GUESSES)) {
    if (length(candidates_idx) == 0) break

    entropy_before <- entropy(length(candidates_idx))

    # guess selection (valid)
    if (!invalid) {
      lf <- letter_freq(candidates_idx, guesses_split)
      scores <- vapply(
        candidates_idx,
        \(i) score_valid_guess(guesses_split[[i]], state, lf),
        numeric(1)
      )
      guess_idx <- candidates_idx[which.max(scores)]

      # occasional human mistake
      if (runif(1) < 0.15) {
        guess_idx <- sample(candidates_idx, 1)
      }
    }
  }
}
```



```

} else {
  # Bot strategy: use known optimal first guess, then maximize entropy
  if (g == 1) {
    guess_idx <- match(OPTIMAL_FIRST_GUESS, word_list)
  } else {
    ent <- compute_all_entropies(candidates_idx, candidates_matrix, guesses_split)
    guess_idx <- candidates_idx[which.max(ent)]
  }
}

feedback <- get_feedback_chars(
  guesses_split[[guess_idx]],
  guesses_split[[solution_idx]]
)

state <- update_state(state, guesses_split[[guess_idx]], feedback)

# Filter candidates and update matrix
keep <- filter_candidates(
  candidates_idx, guess_idx, feedback, guesses_split, candidates_matrix
)
keep_mask <- candidates_idx %in% keep
candidates_idx <- keep
candidates_matrix <- candidates_matrix[keep_mask, , drop = FALSE]

history[[g]] <- list(
  guess = word_list[guess_idx],
  feedback = feedback,
  entropy_before = entropy_before
)

if (guess_idx == solution_idx) break
}

extract_features(history)
}

simulate_valid_game <- function(solution, word_list, guesses_split) {
  simulate_game(
    solution_idx = match(solution, word_list),
    guesses_split = guesses_split,
    word_list = word_list,
    invalid = FALSE
  )
}

simulate_invalid_game <- function(solution, word_list, guesses_split) {
  simulate_game(
    solution_idx = match(solution, word_list),
    guesses_split = guesses_split,
    word_list = word_list,
    invalid = TRUE
  )
}

```

```
)  
}
```

With the help of parallel computing, 1000 rows of Wordle games from each set of conditions were simulated, labeled as “valid”/“invalid” and combined into one .csv file.

Real Wordle Dataset

While the simulated data provides labeled examples of assisted and unassisted play, it’s important that the models can be applied to real data. To accomplish this, I used a publicly available Wordle dataset from Kaggle containing millions of historical Wordle games (Scarcalvetsis. (2022). Wordle games [Data set]. Kaggle <https://www.kaggle.com/datasets/scarcalvetsis/wordle-games>). This Kaggle set is the result of the Twitter API scraping 6.92 million tweets between January 1st 2022 and November 15th 2022.

For the sake of this project, we will be training our models on the synthetic data, then use the models on a randomly sampled subset of this public Kaggle data.

The Kaggle dataset itself has the following variables:

Game: Contains the game ID used by Wordle to identify the target word of the day (ex: '12843').

Trial: Contains the number of attempts that the player made until winning or losing (1-6).

Username: Contains a random integer which uniquely identify each of the different players in the dataset.

processed_text: Contains the resulting games, with colored squares representing the obtained values in a game.

target: Contains the word meant to be guessed by the player (ex: 'apple').

We will only need the emoji summaries from the `processed_text` column, and instead we will derive the following variables:

n_guesses: the total number of guesses made by the player before solving the puzzle or reaching the maximum allowed attempts.

entropy_start: the Shannon entropy of the candidate solution set at the beginning of the game, representing the initial uncertainty about the target word.

entropy_end: the entropy of the remaining candidate solution set after the final guess, reflecting how much uncertainty remained at the end of the game.

mean_entropy_drop: the average reduction in entropy between consecutive guesses, measuring the typical amount of information gained per guess.

max_entropy_drop: the largest single reduction in entropy observed during the game, representing the most informative guess made by the player.

early_entropy_drop: the average entropy reduction during the first two guesses, capturing how quickly the player reduces uncertainty early in the game.

monotone_entropy_drop: T/F, indicates whether or not entropy consistently decreased over the course of the game.

green_fraction: the average proportion of green tiles (correct letter and correct position) across guesses in a game.

yellow_fraction: the average proportion of yellow tiles (correct letter but incorrect position) across guesses in a game.

These same variables were generated in the synthetic dataset so that the resulting feature distributions would be directly comparable to those extracted from the real gameplay data.

A Note About Entropy Entropy is a concept that will become incredibly valuable as a variable in this project. In this context, entropy comes from information theory and represents the amount of uncertainty

about the correct answer. For our purposes, it can be thought of as the amount of “wiggle room” remaining when trying to determine the Wordle solution.

At any point in the game, there are several words that could still be the correct answer based on the feedback received so far. If many possible words remain, there is still a lot of uncertainty. If only a few possible words remain, the uncertainty is much smaller.

Tracking how quickly this “wiggle room” shrinks throughout the game helps describe how efficiently a player is narrowing down the solution. Solver-assisted strategies tend to reduce the number of possible words very quickly, while more typical human play often reduces this uncertainty more gradually. Because of this, changes in entropy over the course of a game can provide useful information for distinguishing between assisted and unassisted play.

Here is the code that derives these features from the sampled real data:

```
#extracting same features from real data

parse_processed_text <- function(txt) {
  txt <- gsub("\\[|\\]|'|\\\"", "", txt)
  rows <- unlist(strsplit(txt, "\\s+"))
  rows <- rows[nzchar(rows)]

  lapply(rows, function(row) {
    chars <- strsplit(row, "")[[1]]
    as.integer(dplyr::recode(chars,
                             " " = 0L,
                             " " = 0L,
                             " " = 1L,
                             " " = 2L
    ))
  })
}

set.seed(212026)
real_data <- wordle_games %>% sample_n(size = 1000, replace = FALSE)

build_real_history <- function(processed_text,
                              target,
                              word_list,
                              guesses_split) {

  feedback_list <- parse_processed_text(processed_text)

  solution_idx <- match(tolower(target), word_list)
  if (is.na(solution_idx)) return(NULL)

  # Build full candidate matrix once
  all_chars_matrix <- do.call(rbind, guesses_split)
  candidates_idx <- seq_along(word_list)
  candidates_matrix <- all_chars_matrix

  history <- list()

  for (g in seq_along(feedback_list)) {

    if (length(candidates_idx) == 0) break
```

```

entropy_before <- entropy(length(candidates_idx))
feedback <- feedback_list[[g]]

# Filter exactly like synthetic
fb_matrix <- get_feedback_matrix(
  guesses_split[[solution_idx]],
  candidates_matrix
)

keep <- apply(fb_matrix, 1, function(row) all(row == feedback))

candidates_idx <- candidates_idx[keep]
candidates_matrix <- candidates_matrix[keep, , drop = FALSE]

history[[g]] <- list(
  entropy_before = entropy_before,
  feedback = feedback
)

if (all(feedback == 2)) break
}

history
}

extract_real_features <- function(processed_text,
                                target,
                                word_list,
                                guesses_split) {

  history <- build_real_history(
    processed_text = processed_text,
    target = target,
    word_list = word_list,
    guesses_split = guesses_split
  )

  if (is.null(history) || length(history) == 0) return(NULL)

  extract_features(history)
}

fb_matrix <- parse_processed_text(real_data$processed_text)

real_features <- real_data %>%
  mutate(features = purrr::pmap(
    list(processed_text, target),
    extract_real_features,
    word_list = guesses,
    guesses_split = guesses_split
  )) %>%
  tidyr::unnest(features)

```

```

real_features$valid <- NA
real_features$source <- "real"

real_features <- real_features %>%
  select(all_of(intersect(names(real_features), names(synthetic_data))))

write_csv(real_features, "real_features.csv")

```

Exploratory Data Analysis

Now that we have generated our synthetic data, we'll do some basic exploratory data analysis for the synthetic data, just to make sure that our assumptions are represented in the data.

```

library(tidyverse)
library(dplyr)
library(purrr)
library(furrr)
library(ggplot2)
library(yardstick)
library(tune)
library(knitr)
library(kableExtra)
library(tidymodels)
library(readr)
library(patchwork)

# EDA plots
synthetic_data <- read_csv("synthetic_data.csv")

palette <- c(
  "synthetic_invalid" = "#c8b653",
  "synthetic_valid"   = "#6ca965"
)

p1 <- ggplot(synthetic_data, aes(x = source, y = mean_entropy_drop, fill = source)) +
  geom_violin(trim = FALSE, alpha = 0.6) +
  geom_boxplot(width = 0.15, outlier.shape = NA) +
  labs(title = "Mean entropy drop per guess", y = "Mean entropy drop (bits)", x = NULL) +
  scale_fill_manual(values = palette) +
  theme_minimal()

p2 <- ggplot(synthetic_data, aes(x = source, y = max_entropy_drop, fill = source)) +
  geom_boxplot(alpha = 0.7) +
  labs(title = "Maximum single-step entropy drop", y = "Max entropy drop (bits)", x = NULL) +
  scale_fill_manual(values = palette) +
  theme_minimal()

p3 <- ggplot(synthetic_data, aes(x = source, y = early_entropy_drop, fill = source)) +
  geom_violin(trim = TRUE, alpha = 0.6) +
  geom_boxplot(width = 0.15, outlier.shape = NA) +
  scale_fill_manual(values = palette) +

```

```

labs(title = "Early-game entropy reduction", subtitle = "Mean entropy drop over first two guesses", y
theme_minimal()

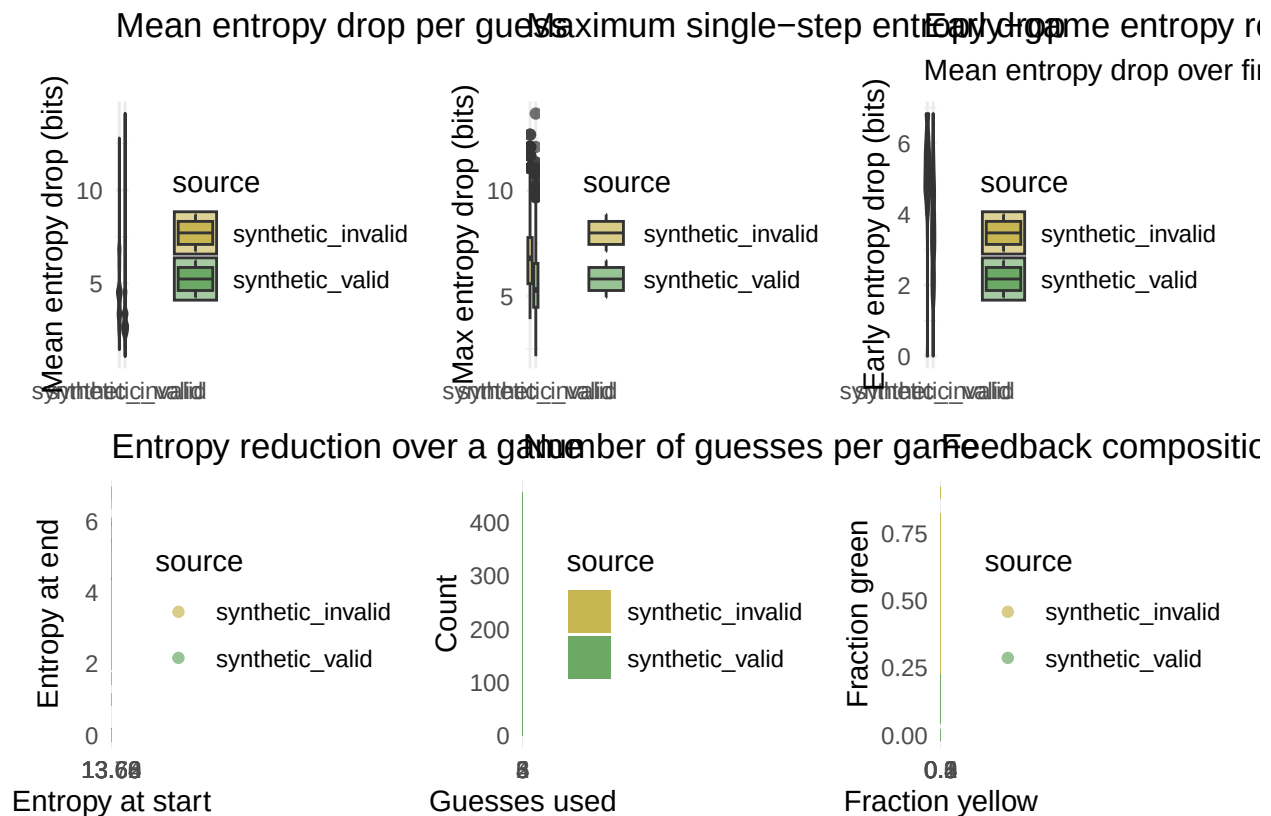
p4 <- ggplot(synthetic_data, aes(x = entropy_start, y = entropy_end, color = source)) +
  geom_point(alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  scale_color_manual(values = palette) +
  labs(title = "Entropy reduction over a game", x = "Entropy at start", y = "Entropy at end") +
  theme_minimal()

p5 <- ggplot(synthetic_data, aes(x = n_guesses, fill = source)) +
  geom_bar(position = "dodge") +
  scale_fill_manual(values = palette) +
  labs(title = "Number of guesses per game", x = "Guesses used", y = "Count") +
  theme_minimal()

p6 <- ggplot(synthetic_data,
  aes(x = yellow_fraction, y = green_fraction, color = source)) +
  geom_point(alpha = 0.7) +
  scale_color_manual(values = palette) +
  labs(title = "Feedback composition by game", x = "Fraction yellow", y = "Fraction green") +
  theme_minimal()

(p1 | p2 | p3) /
(p4 | p5 | p6)

```



Off the bat, we can conclude the following from the synthetic plots:

- 1) The invalid games have a higher average drop in entropy per guess than the valid ones.
- 2) The invalid games have a higher maximum drop in entropy per guess than the valid ones.
- 3) The invalid games have a higher early-game drop in entropy per guess than the valid ones.
- 4) The invalid games are generally better at completely reducing entropy (in other words, they have a higher solve rate).
- 5) The valid games have a higher guess count than the invalid games.
- 6) Valid games have a lower green fraction, while invalid games have a higher green fraction and a lower yellow fraction.

By and large, our assumptions have been successfully represented in the synthetic data, and the differences aren't significant between valid and invalid, which is good since it adds nuance and will also prevent our models from dealing with any clean separation later on in the training process.

For the sake of comparison, here are the same plots, using the random sample from the real data:

```
# EDA plots
real_features <- read_csv("real_features.csv")

p7 <- ggplot(real_features, aes(x = source, y = mean_entropy_drop)) +
  geom_boxplot(width = 0.15, outlier.shape = NA) +
  labs(
    title = "Mean entropy drop per guess",
    y = "Mean entropy drop (bits)",
    x = NULL
  ) +
  theme_minimal()

p8 <- ggplot(real_features, aes(x = source, y = max_entropy_drop)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title = "Maximum single-step entropy drop",
    y = "Max entropy drop (bits)",
    x = NULL
  ) +
  theme_minimal()

p9 <- ggplot(real_features, aes(x = source, y = early_entropy_drop)) +
  geom_boxplot(width = 0.15, outlier.shape = NA) +
  labs(
    title = "Early-game entropy reduction",
    subtitle = "Mean entropy drop over first two guesses",
    y = "Early entropy drop (bits)",
    x = NULL
  ) +
  theme_minimal()

p10 <- ggplot(real_features, aes(x = entropy_start, y = entropy_end)) +
  geom_point(alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  labs(
    title = "Entropy reduction over a game",
```

```

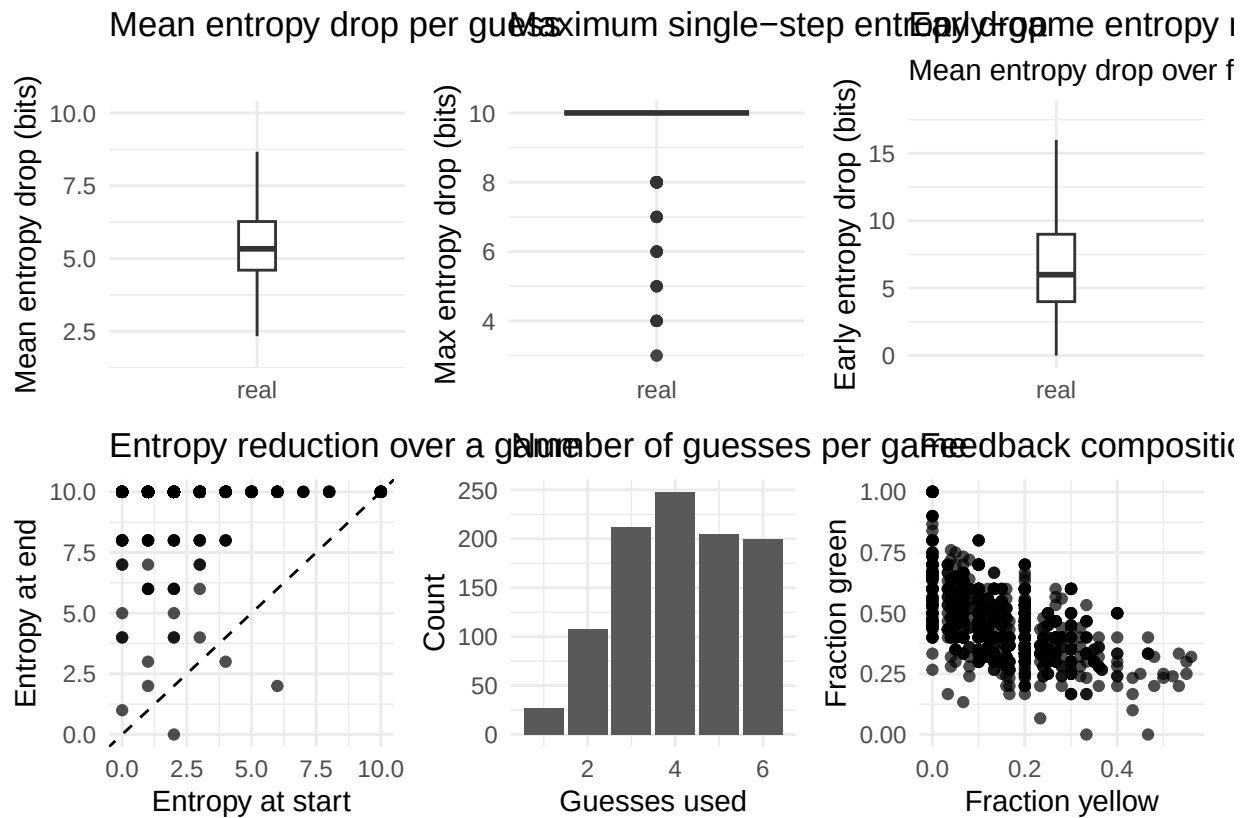
    x = "Entropy at start",
    y = "Entropy at end"
  ) +
  theme_minimal()

p11 <- ggplot(real_features, aes(x = n_guesses)) +
  geom_bar(position = "dodge") +
  labs(
    title = "Number of guesses per game",
    x = "Guesses used",
    y = "Count"
  ) +
  theme_minimal()

p12 <- ggplot(real_features,
  aes(x = yellow_fraction, y = green_fraction)) +
  geom_point(alpha = 0.7) +
  labs(
    title = "Feedback composition by game",
    x = "Fraction yellow",
    y = "Fraction green"
  ) +
  theme_minimal()

(p7 | p8 | p9) /
(p10 | p11 | p12)

```

We see somewhat similar results in mean entropy drop, as well as a greater range in values in the guess count and feedback composition by game.

One important note: entropy values of the real data are approximated as sums of feedback codes (green=2, yellow=1), serving as proxies for information gain since exact entropy requires guess and candidate data (which is not available from emojis). This won't be an issue for the machine learning modeling later on (since the models will be trained on synthetic data), but it does lead to some funky early visualizations, especially in the maximum entropy drop, and entropy reduction over a game.

As for missing data, due to the synthetic nature of the data, **missing data does not exist** (nor does any missing data exist in the real Kaggle dataset).

The Machine Learning Pipeline

Now that we have sufficiently generated and explored our data, it's time for the fun part! We will split the synthetic dataset into training and testing sets, identify the best model from a variety of options based on the testing set, and then use the best-performing model on the real Kaggle set.

Train/Test Split

In this step, we set a random seed (for reproducibility), coerce the outcome variable (`valid`) into a factor, split the synthetic data into training and testing sets using a 70/30 split; this allows 70% of the data to train the models sufficiently, while 30% provides a robust test set. Given the moderate size of the dataset, this is a fine proportion to avoid overfitting.

We will also use k-fold cross validation to help with the issue of imbalanced data, stratifying on our response variable, `valid`.

```

#model time

set.seed(2232026)

#factor-ing outcome variable
synthetic_data <- synthetic_data %>%
  mutate(valid = factor(valid)) %>%
  select(-"...1")

#70/30 split, stratifying on outcome variable, k-fold CV
split <- initial_split(
  synthetic_data,
  prop = 0.7,
  strata = valid
)

train_data <- training(split)
train_data$valid <- factor(train_data$valid)
test_data <- testing(split)
test_data$valid <- factor(test_data$valid)

folds <- vfold_cv(
  train_data,
  v = 5,
  strata = valid
)

```

Recipe + Model Building

Upon splitting and stratifying the data, we can now start building the recipe for our models. Looking further at `synthetic_data` itself, `monotone_entropy` happens to be `TRUE` for every single row, so it will be removed from the dataset entirely since it isn't a particularly informative variable, much like `source`, which blatantly spoils whether or not the row is valid or not, and will also be removed so that the models do not cleanly separate.

The model is set up so that the outcome variable `valid` is predicted as a function of the other predictor variables. Any categorical features are converted to dummy variables so the models can use them (for the record, all of the remaining variables are numeric; the line is included for the sake of hygiene). Numeric features are centered and scaled, which is necessary for models like logistic regression and elastic net. Missing values are imputed using the median for numeric features and the mode for categorical features (there are no missing values, so again, the steps realistically do nothing, but they are included for the sake of good practice). This recipe ensures that all models receive a clean, appropriately processed dataset.

```

train_data <- train_data %>% select(-monotone_entropy, -source)
test_data <- test_data %>% select(-monotone_entropy, -source)

#recipe

rec <- recipe(valid ~ ., data = train_data) %>%
  step_zv(all_predictors()) %>% # removes constant columns
  step_dummy(all_nominal_predictors()) %>%
  step_impute_median(all_numeric_predictors()) %>%
  step_impute_mode(all_nominal_predictors()) %>%

```

```
step_normalize(all_numeric_predictors())
```

We will fit four different classification models: standard logistic regression, elastic net logistic regression, random forest, and gradient boosted trees. Each model predicts the outcome (`valid`) as a function of the predictor features.

For models that can be tuned (elastic net, random forest, and gradient boosted trees), we define reasonable hyperparameter ranges and perform 5-fold cross-validation to select the best parameter combinations. Logistic regression, which does not require hyperparameter tuning, is evaluated directly with cross-validation.

This approach ensures that all models are appropriate for a binary classification problem, and any model with tunable parameters is properly tuned using cross-validation.

```
#engines
log_spec <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")

enet_spec <- logistic_reg(
  penalty = tune(),
  mixture = tune()
) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

rf_spec <- rand_forest(
  mtry = tune(),
  trees = 500,
  min_n = tune()
) %>%
  set_engine("ranger") %>%
  set_mode("classification")

xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  min_n = tune()
) %>%
  set_engine("xgboost") %>%
  set_mode("classification")

#workflows
log_wf <- workflow() %>% add_model(log_spec) %>% add_recipe(rec)
enet_wf <- workflow() %>% add_model(enet_spec) %>% add_recipe(rec)
rf_wf <- workflow() %>% add_model(rf_spec) %>% add_recipe(rec)
xgb_wf <- workflow() %>% add_model(xgb_spec) %>% add_recipe(rec)

#tuning grids
enet_grid <- grid_regular(
  penalty(range = c(-4, 1)),
  mixture(range = c(0, 1)),
  levels = 5
)
```

```

num_pred <- ncol(train_data) - 1 - 2 # minus outcome + removed cols
rf_grid <- grid_regular(
  mtry(range = c(1, num_pred)),
  min_n(range = c(2, 10)),
  levels = 5
)

xgb_grid <- grid_regular(
  trees(range = c(300, 1000)),
  tree_depth(range = c(3, 8)),
  learn_rate(range = c(-3, -0.5)),
  min_n(range = c(2, 10)),
  levels = 3
)

#tuning
my_metrics <- yardstick::metric_set(
  yardstick::roc_auc,
  yardstick::accuracy
)

#log = no tuning
log_res <- fit_resamples(
  log_wf,
  resamples = folds,
  metrics = my_metrics
)

#the rest (yes tuning)
#enet_res <- tune_grid(
#  enet_wf,
#  resamples = folds,
#  grid = enet_grid,
#  metrics = my_metrics
#)

#saveRDS(enet_res, "enet_res.rds")
enet_res <- readRDS("enet_res.rds")

#rf_res <- tune_grid(
#  rf_wf,
#  resamples = folds,
#  grid = rf_grid,
#  metrics = my_metrics
#)

#saveRDS(rf_res, "rf_res.rds")
rf_res <- readRDS("rf_res.rds")

#xgb_res <- tune_grid(
#  xgb_wf,
#  resamples = folds,
#  grid = xgb_grid,

```

```
# metrics = my_metrics
#)

#saveRDS(xgb_res, "xgb_res.rds")
xgb_res <- readRDS("xgb_res.rds")
```

The hyperparameter ranges for elastic net, random forest, and XGBoost were chosen to balance coverage of the parameter space with computational feasibility.

For elastic net, the penalty (`lambda`) spans 10^{-4} to 10^{-1} , allowing the model to explore both minimal and strong regularization, while the mixture (`alpha`) covers the full ridge-to-lasso spectrum [0,1].

For random forest, `mtry` ranges from 1 to the number of predictors minus one, ensuring splits can consider both few and many features, and `min_n` ranges from 2 to 10 to allow control over tree depth and prevent overfitting.

For XGBoost, the number of trees (300–1000) and tree depth (3–8) capture a variety of model complexities, while the learning rate from 0.001 to 0.316 ensures both slow and moderately fast updates. Minimum node size (`min_n` = 2–10) further prevents overfitting in both tree-based models. Overall, these ranges are standard for classification problems, cover underfitting and overfitting regimes, and provide the models sufficient flexibility to learn from the data effectively.

Now that we have tuned our models, let’s find out which ones performed the best!

Table 1: Top 10 Elastic Net Hyperparameter Combinations

penalty	mixture	.metric	.estimator	mean	n	std_err	.config
0.002	1.00	roc_auc	binary	0.874	5	0.009	pre0_mod10_post0
0.002	0.50	roc_auc	binary	0.874	5	0.009	pre0_mod08_post0
0.000	0.50	roc_auc	binary	0.874	5	0.009	pre0_mod03_post0
0.002	0.75	roc_auc	binary	0.874	5	0.009	pre0_mod09_post0
0.000	0.75	roc_auc	binary	0.874	5	0.009	pre0_mod04_post0
0.000	1.00	roc_auc	binary	0.874	5	0.009	pre0_mod05_post0
0.000	0.25	roc_auc	binary	0.874	5	0.009	pre0_mod02_post0
0.002	0.25	roc_auc	binary	0.874	5	0.009	pre0_mod07_post0
0.000	0.00	roc_auc	binary	0.872	5	0.009	pre0_mod01_post0
0.002	0.00	roc_auc	binary	0.872	5	0.009	pre0_mod06_post0

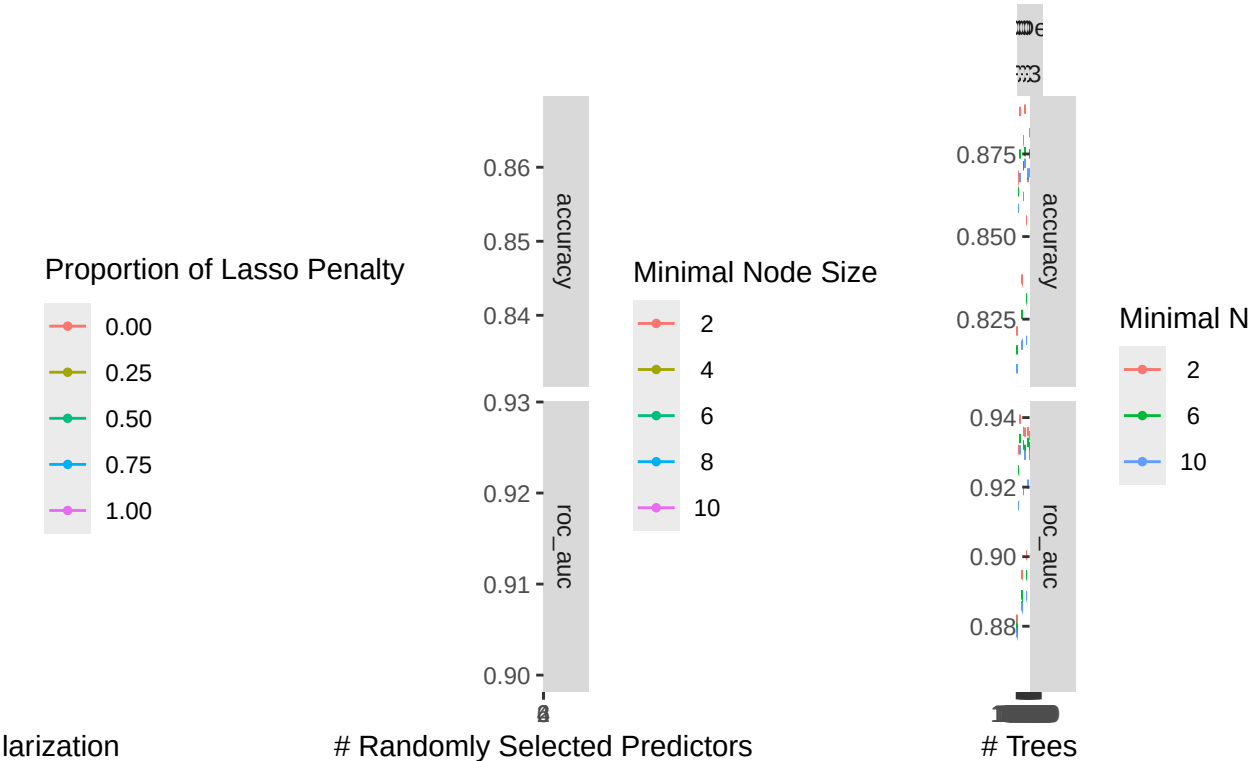
Table 2: Top 10 Random Forest Hyperparameter Combinations

mtry	min_n	.metric	.estimator	mean	n	std_err	.config
6	2	roc_auc	binary	0.929	5	0.003	pre0_mod21_post0
6	4	roc_auc	binary	0.928	5	0.003	pre0_mod22_post0
6	6	roc_auc	binary	0.926	5	0.004	pre0_mod23_post0
6	8	roc_auc	binary	0.925	5	0.004	pre0_mod24_post0
4	2	roc_auc	binary	0.925	5	0.004	pre0_mod16_post0
6	10	roc_auc	binary	0.925	5	0.004	pre0_mod25_post0
4	4	roc_auc	binary	0.923	5	0.004	pre0_mod17_post0
4	6	roc_auc	binary	0.923	5	0.004	pre0_mod18_post0
3	2	roc_auc	binary	0.922	5	0.005	pre0_mod11_post0
4	8	roc_auc	binary	0.921	5	0.004	pre0_mod19_post0

Table 3: Top 10 XGBoost Hyperparameter Combinations

trees	min_n	tree_depth	learn_rate	.metric	.estimator	mean	n	std_err	.config
650	2	3	0.316	roc_auc	binary	0.941	5	0.004	pre0_mod30_post0
300	2	3	0.316	roc_auc	binary	0.940	5	0.003	pre0_mod03_post0
1000	2	3	0.316	roc_auc	binary	0.940	5	0.004	pre0_mod57_post0
300	2	5	0.316	roc_auc	binary	0.937	5	0.004	pre0_mod06_post0
650	2	5	0.316	roc_auc	binary	0.936	5	0.004	pre0_mod33_post0
1000	2	5	0.018	roc_auc	binary	0.936	5	0.005	pre0_mod59_post0
1000	2	8	0.018	roc_auc	binary	0.936	5	0.004	pre0_mod62_post0
1000	2	5	0.316	roc_auc	binary	0.936	5	0.004	pre0_mod60_post0
300	6	3	0.316	roc_auc	binary	0.935	5	0.005	pre0_mod12_post0
650	2	8	0.316	roc_auc	binary	0.935	5	0.003	pre0_mod36_post0

Elastic Net Tuning: ROC AUC across Penalty & Forest Tuning: ROC AUC across mtry



The elastic net model achieved a maximum ROC AUC of approximately 0.874, with many hyperparameter combinations performing nearly identically. Very small penalties (0–0.002) consistently produced the best performance, suggesting that little regularization is needed for this feature set. Performance dropped sharply as the penalty increased, particularly when the model relied more heavily on the LASSO component. The mixture parameter (proportion of LASSO vs. ridge penalty) had minimal impact when the penalty was small, indicating that the model’s predictive signal is relatively stable across different regularization structures. Overall, elastic net performed moderately well but was outperformed by the tree-based models.

The random forest model achieved a higher maximum ROC AUC of approximately 0.929, indicating stronger predictive performance than elastic net. Performance generally improved as the number of randomly selected predictors (mtry) increased, with the best results occurring at mtry = 6. Smaller minimal node sizes (min_n = 2) produced the best results, suggesting that deeper trees capturing more granular patterns improved classification performance. However, differences among the top hyperparameter combinations were relatively

small, indicating that the model's performance is fairly robust to moderate changes in hyperparameters.

The gradient boosted tree model produced the best overall performance, achieving a maximum ROC AUC of approximately 0.941, which is incredible. The best-performing configurations used a moderate tree depth (3) and a high learning rate (~0.316), indicating that relatively shallow trees combined with aggressive boosting updates were effective for this problem. Increasing the number of trees beyond roughly 300–650 provided minimal additional improvement, suggesting diminishing returns with larger ensembles. As with the random forest model, smaller minimum node sizes (`min_n = 2`) tended to produce the strongest results. Overall, XGBoost provided the highest predictive accuracy and most consistent performance across tuning configurations.

Model Fitting

```
best_enet <- select_best(enet_res, metric = "roc_auc")
best_rf   <- select_best(rf_res, metric = "roc_auc")
best_xgb  <- select_best(xgb_res, metric = "roc_auc")

final_enet <- finalize_workflow(enet_wf, best_enet)
final_rf   <- finalize_workflow(rf_wf, best_rf)
final_xgb  <- finalize_workflow(xgb_wf, best_xgb)

#fitting
log_fit <- fit(log_wf, data = train_data)
enet_fit <- fit(final_enet, data = train_data)
rf_fit <- fit(final_rf, data = train_data)
xgb_fit <- fit(final_xgb, data = train_data)

#evaluating on synthetic test set
test_preds_xgb <- augment(xgb_fit, test_data)

test_preds_log <- augment(log_fit, test_data)

test_preds_enet <- augment(enet_fit, test_data)

test_preds_rf <- augment(rf_fit, test_data)
```

Now to find out which one performed the best:

```
metric_set(roc_auc, accuracy)(data = test_preds_log, truth = valid, estimate = .pred_class, .pred_1)

## # A tibble: 2 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>      <dbl>
## 1 accuracy binary      0.795
## 2 roc_auc  binary      0.123

metric_set(roc_auc, accuracy)(test_preds_enet, truth = valid, estimate = .pred_class, .pred_1)

## # A tibble: 2 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>      <dbl>
## 1 accuracy binary      0.8
## 2 roc_auc  binary      0.124
```

```
metric_set(roc_auc, accuracy)(test_preds_rf, truth = valid, estimate= .pred_class, .pred_1)

## # A tibble: 2 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 accuracy binary      0.863
## 2 roc_auc  binary      0.0742

metric_set(roc_auc, accuracy)(test_preds_xgb, truth = valid, estimate = .pred_class, .pred_1) #best one

## # A tibble: 2 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 accuracy binary      0.895
## 2 roc_auc  binary      0.0548
```

With accuracies of 0.895 and 0.8633333, it looks like gradient boosting performed the best, followed by the random forest not too far behind.

“Testing”

Now that we’ve identified our best performing models, we can test them on our real Kaggle set:

```
#real test data
# Predict on real features using each fitted model
pred_rf <- predict(rf_fit, real_features, type = "prob") %>% bind_cols(real_features)

## New names:
## * `...1` -> `...3`

pred_xgb <- predict(xgb_fit, real_features, type = "prob") %>% bind_cols(real_features)

## New names:
## * `...1` -> `...3`
```

Since the real data is unlabeled, we won’t be able to have a testing error from the models. Instead, we can investigate the predictions and see what trends are identified.

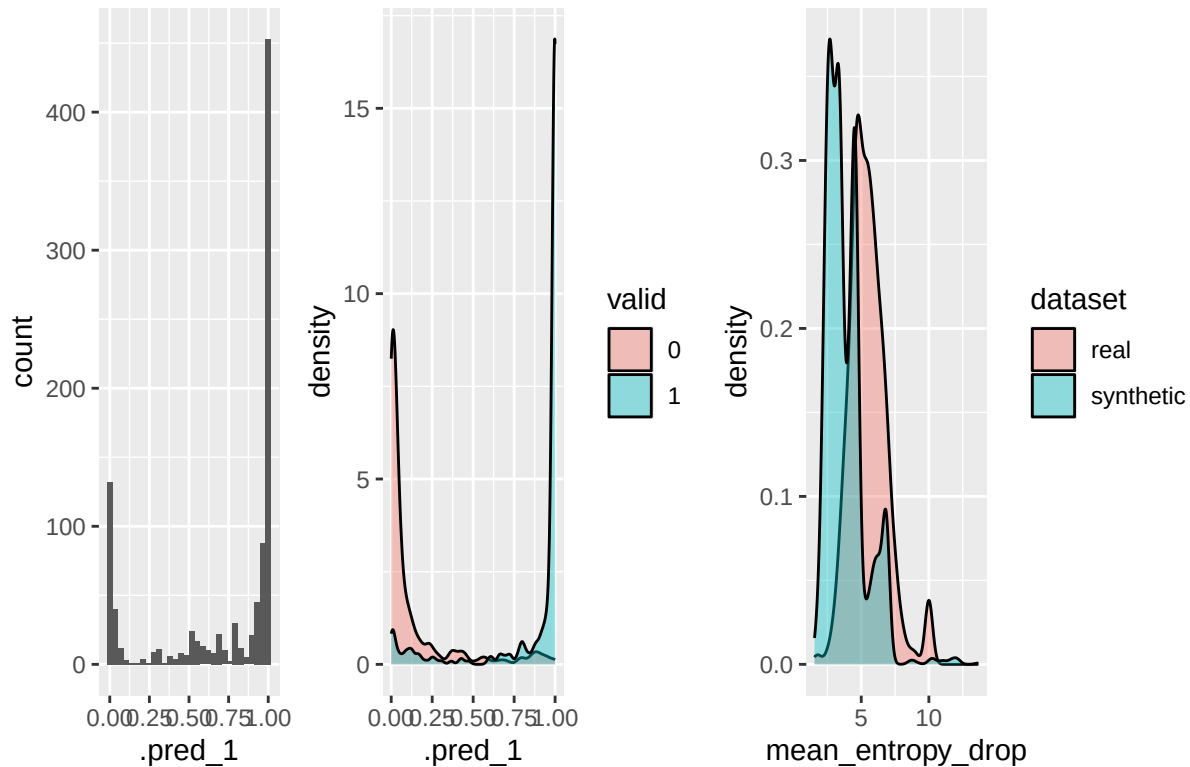
```
p16 <- ggplot(pred_xgb, aes(x = .pred_1)) +
  geom_histogram(bins = 30) +
  labs(title = "Predicted Probability of 'Valid' on Real Data")

p17 <- ggplot(test_preds_xgb, aes(x = .pred_1, fill = valid)) +
  geom_density(alpha = 0.4) +
  labs(title = "Predicted Probability Distributions by True Class")

p18 <- ggplot(bind_rows(
  synthetic_data %>% mutate(dataset = "synthetic"),
  real_features %>% mutate(dataset = "real")
),
  aes(x = mean_entropy_drop, fill = dataset)) +
  geom_density(alpha = 0.4)

(p16 | p17 | p18)
```


Predicted Probability of Invalid Play Distributions by True Class



From the first plot, we see somewhat of a bimodal distribution; this implies that the model confidently separates two groups of play behaviors, suggesting that some real games resemble the synthetic “invalid” patterns (indicating that some percentage of the real games perhaps cheated). The peak towards 1 implies that most real games resemble the patterns learned from valid play in the synthetic dataset (a.k.a. most people were honest; a welcome and reassuring reflection of our society these days).

The plot comparing feature distributions shows pretty high peaks at 0 and 1; this shows that although we can’t quantify error, the model generally does a pretty good job at differentiating valid from invalid, showing strong class separation on the held-out synthetic test data and very little overlap among the distributions. This is encouraging, and likely means that the model generalizes well for real data, and that synthetic data likely captured the important feature patterns well.

From the third plot, we see that distribution of mean entropy drop differs between the synthetic and real datasets. Synthetic games tend to have lower values and a tighter range, while real games are shifted toward higher values and show greater variability. This difference is expected because the real data uses a proxy measure based on feedback codes rather than the true entropy calculation used for synthetic data. As a result, the plot should be interpreted as a rough qualitative comparison rather than a direct, scale-matched comparison of information gain. That said, the difference is still helpful for the sake of sanity-checking; upon further considerations, it makes sense that the synthetic data has a more narrow distribution (a reflection of the more rigid algorithm parameters), while the real data is more spread out, reflecting variety among the millions of games played by real people.

Some other things we can look at:

1) Estimating the proportion of potentially invalid real games

```
invalid_rate <- pred_xgb %>%
  summarize(prop_invalid = mean(.pred_1 < 0.5))

invalid_rate %>%
```

```
mutate(prop_invalid = round(prop_invalid * 100, 2))
```

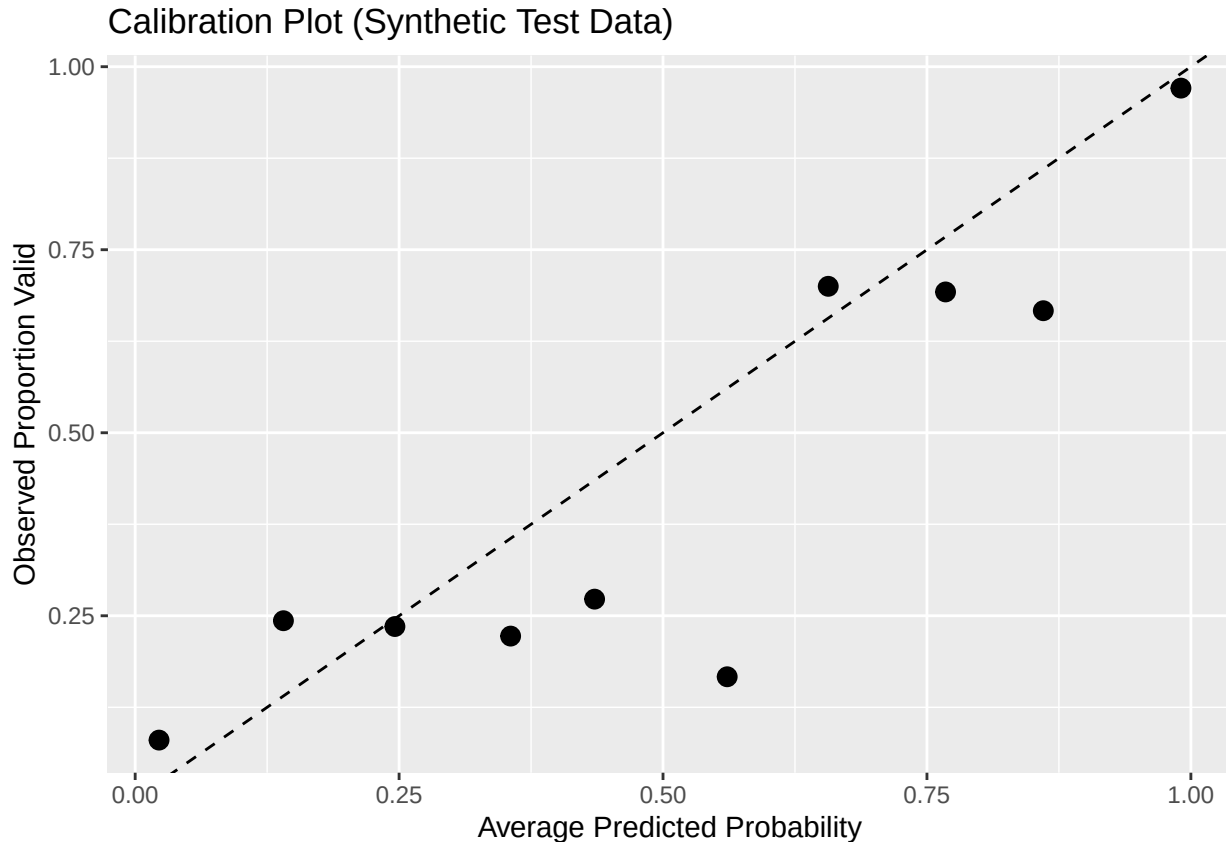
```
## # A tibble: 1 x 1
##   prop_invalid
##         <dbl>
## 1         24
```

Based on the model's predictions, approximately 24% of real games resemble the synthetic patterns associated with invalid play.

2) Calibration of the synthetic test set

```
calibration_data <- test_preds_xgb %>%
  mutate(prob_bin = cut(.pred_1, breaks = seq(0,1,by=0.1), include.lowest = TRUE)) %>%
  group_by(prob_bin) %>%
  summarize(
    avg_pred = mean(.pred_1),
    actual_rate = mean(valid == 1)
  )

ggplot(calibration_data, aes(x = avg_pred, y = actual_rate)) +
  geom_point(size = 3) +
  geom_abline(linetype = "dashed") +
  labs(
    title = "Calibration Plot (Synthetic Test Data)",
    x = "Average Predicted Probability",
    y = "Observed Proportion Valid"
  )
```



3) Checking out extremes

Table 4: Top 10 XGBoost Valid Predictions

```
kable(pred_xgb %>%
  arrange(desc(.pred_1)) %>%
  select(.pred_1, everything()) %>%
  head(10), digits = 3, caption = "Top 10 XGBoost Invalid Predictions")
```

.pred_0	.pred_03	n_guesses	entropy_start	pymc_entropy	mean_entropy	entropy_diff	entropy_diff_norm	entropy_diff_log	entropy_diff_log_norm	yellow_frac	valid	source	
1	0	5680926	5	1	10	2.80	10	2	0	0.200	0.160	NA	real
1	0	2211672	4	0	10	3.25	10	0	0	0.250	0.150	NA	real
1	0	566715	5	0	10	3.20	10	1	0	0.240	0.160	NA	real
1	0	466498	6	0	10	3.50	10	0	0	0.267	0.167	NA	real
1	0	2555436	6	2	10	3.00	10	2	0	0.200	0.200	NA	real
1	0	3145407	5	0	10	3.60	10	1	0	0.280	0.160	NA	real
1	0	6628746	5	0	10	3.60	10	2	0	0.280	0.160	NA	real
1	0	5609927	5	1	10	3.60	10	2	0	0.280	0.160	NA	real
1	0	1297361	5	1	10	3.60	10	2	0	0.280	0.160	NA	real
1	0	3667518	5	0	10	3.60	10	1	0	0.280	0.160	NA	real

27

Let's identify the most important variables from the best performing gradient boosted model using the `vip()` function (since `XGBoost` has importance built-in):

```
library(vip)
```

```
##
```

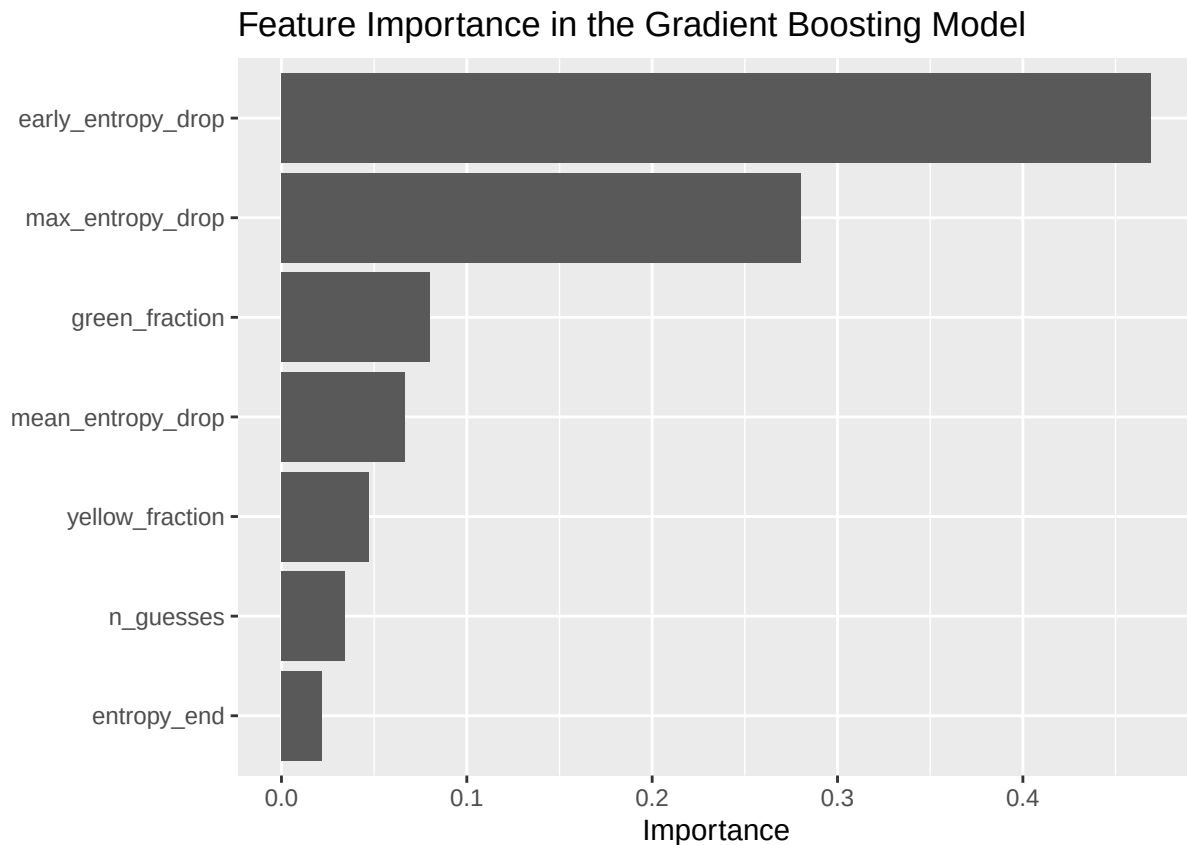
```
## Attaching package: 'vip'
```

```
## The following object is masked from 'package:utils':
```

```
##
```

```
## vi
```

```
vip(xgb_fit) + ggtitle("Feature Importance in the Gradient Boosting Model")
```



From the plot, we can see that the model's predictions are driven mostly by the entropy-related features and the proportion of green letters. In other words, to intuit whether a game might be invalid, the most telling signs are sudden reductions in the number of possible solutions. Personally, I'm fairly surprised that `n_guesses` was towards the bottom in importance as a predictor; my go-to for calling people out was always solving the game within two or three guesses.

Overall, although we couldn't get a definitive testing error from unlabeled data, the prediction distributions, estimated rate of potentially invalid games, calibration behavior on synthetic test data, and feature importance analysis all suggest that the models trained on synthetic data capture meaningful gameplay patterns and produce sensible predictions when applied to real Wordle data.

Conclusion

This project explored whether Wordle emoji summaries can distinguish assisted from unassisted play using machine learning. Because real games lack labels, a synthetic dataset was generated by simulating solver-

based (assisted) and heuristic-based (human-like) strategies, from which gameplay features, mainly entropy reduction and feedback composition, were extracted. Exploratory analysis confirmed meaningful differences between valid and invalid games, enabling the training and evaluation of multiple classifiers, including logistic regression, elastic net, random forest, and gradient boosted trees. Gradient boosting performed best, with feature importance highlighting entropy-based metrics and green-tile fractions. Applied to the real Kaggle dataset, in lieu of producing a definitive testing error, we did some further exploration to investigate the model's predictions. The model produced a bimodal probability distribution, estimating that about 24% of games resembled the assisted strategy, demonstrating that the model can detect gameplay patterns differing from typical human play without implying cheating.

There are several opportunities for future work that could improve this approach. One limitation of the current project is that the synthetic simulation strategies are relatively simple approximations of human and solver behavior. Future work could incorporate more realistic models of human guessing behavior, including multiple common opening words and more refined "imperfect reasoning". I would also incorporate color conversion rates (ex: grey -> yellow, yellow -> green, etc.) as predictors, and scale the synthetic simulations beyond 2000 rows of data. Additionally, the real dataset only contains emoji feedback and not the original guesses, which required the use of proxy measures for entropy. Access to data containing full guess histories (including words) would allow for more accurate entropy calculations and richer feature engineering. Finally, the real data being unlabeled was a substantial limiting factor; one suggested solution was to have real people (likely my friends) track their games over time and cheat randomly without telling me as a double-blind experiment. Together, these improvements could lead to more robust and interpretable models for understanding strategic behavior in Wordle gameplay.

All in all, I am surprised the models performed as well as they did, and I'm proud of myself for sticking with my passion project, navigating algorithm design, information theory, and parallel processing alongside machine learning, and making it out in one piece. At some point in the future, I hope to eventually deploy this model into an app where emoji summaries can be uploaded and immediately evaluated as valid or not as a way of dipping my toes into app development.

Finally, Greg, if you're reading this: your days are numbered.